



Day 3 Notes



Node & Express Basics – Simple Explanation

★ Today = built our first server, and connected it to our React app from Day 2.

1 Frontend vs Backend

Frontend (React) = what the user SEES and clicks on, runs in the browser.

Backend (Express) = code that runs on a computer/server, and answers requests like "give me the products" or "save this order."

The frontend asks the backend for data. The backend sends it back. This is how almost every real website works.

2 Setting Up a Server

We made a new folder and set up a Node project:

```
npm init -y      → creates package.json (project's ID card)
npm install express → installs the Express library
```

package.json keeps track of our project's name and which libraries (packages) it needs.

3 Our First Server (index.js)

```
const express = require('express')
const app = express()
const PORT = 3000

app.get('/', (req, res) => {
  res.send('Hello from the server!')
})

app.listen(PORT, () => {
  console.log('Server running on http://localhost:3000')
})
```

- `require('express')` → brings in the Express library
- `app.get('/', ...)` → a ROUTE: "when someone visits this address, do this"
- `res.send(...)` → sends text back to whoever asked
- `app.listen(PORT, ...)` → starts the server so it can receive requests

Run it with: `node index.js` — then visit `localhost:3000` in the browser to see it work.

4 *Sending Real Data (JSON)*

A real backend usually sends DATA, not plain text. We added a new route for that:

```
app.get('/api/products', (req, res) => {
  res.json([
    { id: 1, name: 'Blue T-Shirt', price: 19.99 },
    { id: 2, name: 'Red Hoodie', price: 39.99 },
    { id: 3, name: 'Black Cap', price: 12.5 },
  ])
})
```

`res.json(...)` sends data in JSON format — the standard way computers pass data to each other online. Visiting `/api/products` in the browser shows raw data, not a nice page — that's normal, it's meant for code to read, not humans.

5 *CORS — Letting React Talk to Express*

By default, browsers BLOCK one website (React, on port 5173) from asking another (Express, on port 3000) for data. This safety rule is called CORS.

```
npm install cors
const cors = require('cors')
app.use(cors())
```

Adding this tells our server: "It's okay, allow other addresses to ask me for data."

6 *Fetching Data in React*

Back in `App.jsx`, instead of a hardcoded list, we asked the server for real data:

```
import { useState, useEffect } from 'react'

const [products, setProducts] = useState([])

useEffect(() => {
  fetch('http://localhost:3000/api/products')
    .then((res) => res.json())
    .then((data) => setProducts(data))
}, [])
```

- `useState([])` → starts with an empty product list
- `useEffect(..., [])` → runs ONCE, right when the page first loads
- `fetch(...)` → sends a request to our Express server

- `setProducts(data)` → once data arrives, save it, which updates the screen

The rest of the page (the `.map()` rendering from Day 2) didn't need to change at all — it just displays whatever is inside "products", whether that came from a hardcoded array or a real server.



Big takeaway:

Express = backend server that answers requests with data (JSON). CORS = permission that lets our React app talk to our Express server. `fetch` + `useEffect` = how React asks the server for data when the page loads. Together: Frontend (shows data) + Backend (provides data) = a real full-stack app. This exact pattern is what we'll use for our real product catalog later.

— made while building: *AI Virtual Try-On E-commerce App* —